

inc-mb-fidelity-inc___fidelity-chain-f-k4-inc

An AgentMPI run analysis

Abstract

This is the deepest cell of the enforced, incompressible reduction: a serial left fold over eight ranks, seven successive applications, `fold_depth: 7`. It reaches the same 0.385 retention and the same 37 survivors as its depth-2, depth-3 and depth-7 siblings — I compared the identifier sets directly and the symmetric difference is empty — which is the campaign’s central negative result about tree shape. What this cell contributes that none of the others can is the shape of the loss along the way, because the fold advances one rank per hop and every intermediate is in the store. The chain is a clean prefix truncation at every step: at depth 2 it keeps ranks 5 and 6 whole and shaves rank 7 to ten items of twelve; at depth 3 it keeps 4, 5, 6 whole and shaves rank 7 to two; at depth 4 it keeps 3 and 4 whole and shaves rank 5 to one. The reduction never distributes its loss and never corrupts a payload; it writes items in rank order until it stops. And it stops too early. The applications that are forced to drop emit only 300 tokens and 25 items against a 450-token ceiling that holds 38, so a third of the available budget goes unused and the loss is worse than the arithmetic requires. The 0.385 endpoint is therefore reached by two different mechanisms in this campaign — a budget-saturating truncation in the tree cells and an undershooting one here — which agree only because 25 surviving items plus rank 0’s twelve happens to equal the ceiling exactly.

1 What this run is

property	value
run	inc-mb-fidelity-inc___fidelity-chain-f-k4-inc
experiment	-
campaign label	-
declared world size	8
ranks appearing in the log	8
executors	<code>broker</code> $\times$ 8, <code>worker</code> $\times$ 47
events	116
wall time	634.44s
job outcome	completed

2 Headline measurements

metric	value	meaning
achieved parallelism	0.99×	total busy time over wall time
parallel efficiency	12.3%	achieved parallelism per rank
idle fraction	87.7%	rank-seconds paid for and unused
peak ranks busy	1	of 8 in the world
serial fraction of active time	100.0%	time with exactly one rank busy
load imbalance	2.07×	slowest rank’s busy time over the mean
coordination share	53.3%	rank-seconds blocked in collectives, of those available
coordination in flight	100.0%	wall clock with any rank inside a collective
rank reattachments	47	fresh agent processes that took over a rank role
agent calls	7	invocations that reached a model
agent latency p50 / p95	78.53 / 162.20 s	per invocation
messages	7	7 eager, 0 rendezvous
tokens sent	2,448	0 deferred under rendezvous
tokens in / out	4,342 / 2,623	prompt and completion
cost	\$0.0524	0.0200 \$/kTok out

3 What the analysis notices

- **[critical]** At least one rank waited 2s for an agent to claim its task before the broker gave up. That is a pool-sizing failure outside the protocol, not a slow run.
- **[warning]** Concurrency never exceeded one rank despite a world size of 8: this ran serially.
- **[note]** Collectives account for 53.3% of the run’s rank-seconds, so more of the pool’s time went to coordination than to work.
- **[warning]** The cost model’s α and β here are a median fallback, not a fit: the slope was positive but the intercept was negative ($\alpha = -96.63$ s), so the fitted line passes below the origin. That is physically impossible and statistically unremarkable on a narrow token range, and the guard rejects it rather than publish a negative fixed cost, on a fit with $R^2 = 0.473$. Any latency prediction from this run’s calibration is a median, not a model.
- **[note]** Load imbalance is 2.07×: the slowest rank was busy far longer than the mean.
- **[ok]** 47 rank reattachment(s) occurred, up to incarnation 15: agent processes were replaced mid-run while the rank roles, their mailboxes, and their context accounts persisted.
- **[ok]** All 1 checkable collective(s) sent exactly the number of messages their closed-form cost expression predicts.
- **[ok]** All 1 checkable collective(s) also matched the predicted round count.

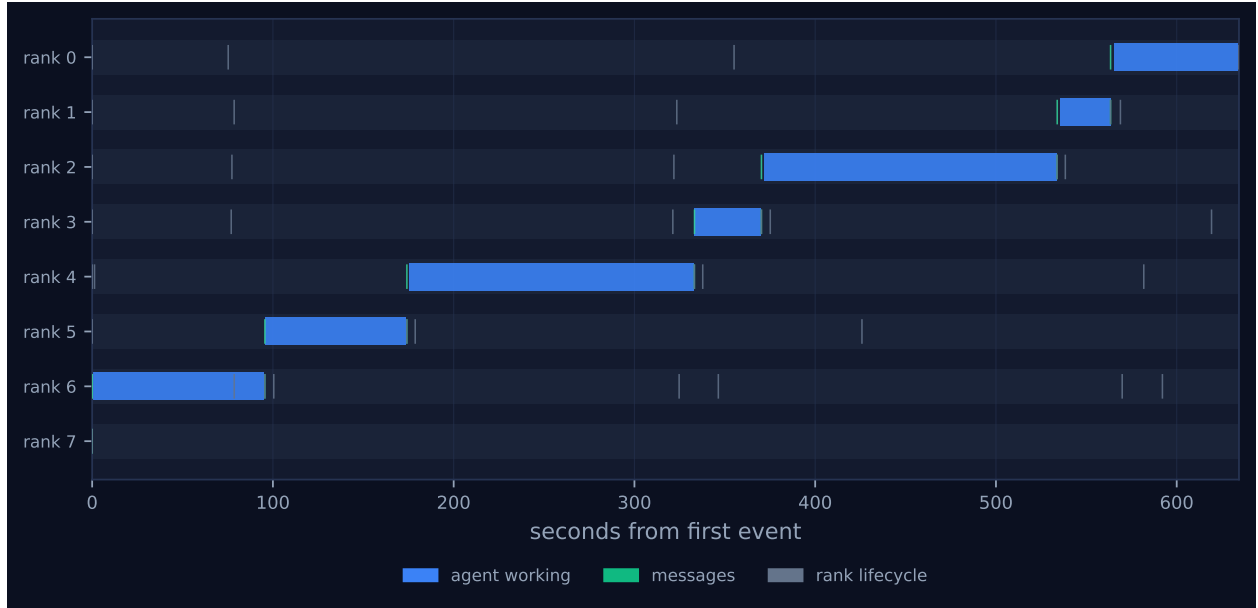


Figure 1: Execution timeline, one lane per rank. Bars are intervals when a rank held a task; ticks are messages; diamonds are window operations, lifecycle transitions, failures, and recovery actions. Drawn in the trace viewer's palette so the same shapes read the same way in both.

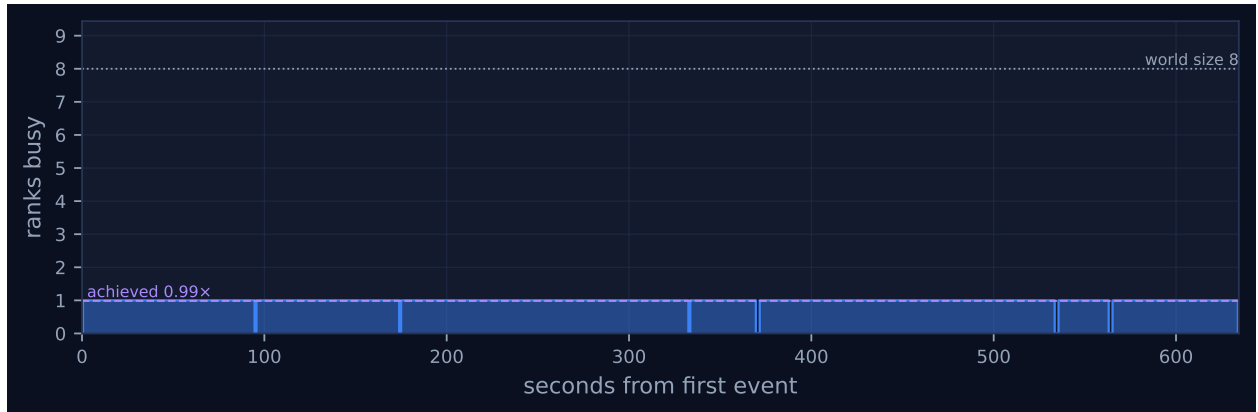


Figure 2: Ranks simultaneously busy over time. The dotted line is the declared world size and the dashed line the achieved parallelism; the gap between them is what the run paid for and did not use. Faint vertical lines mark collective invocations.

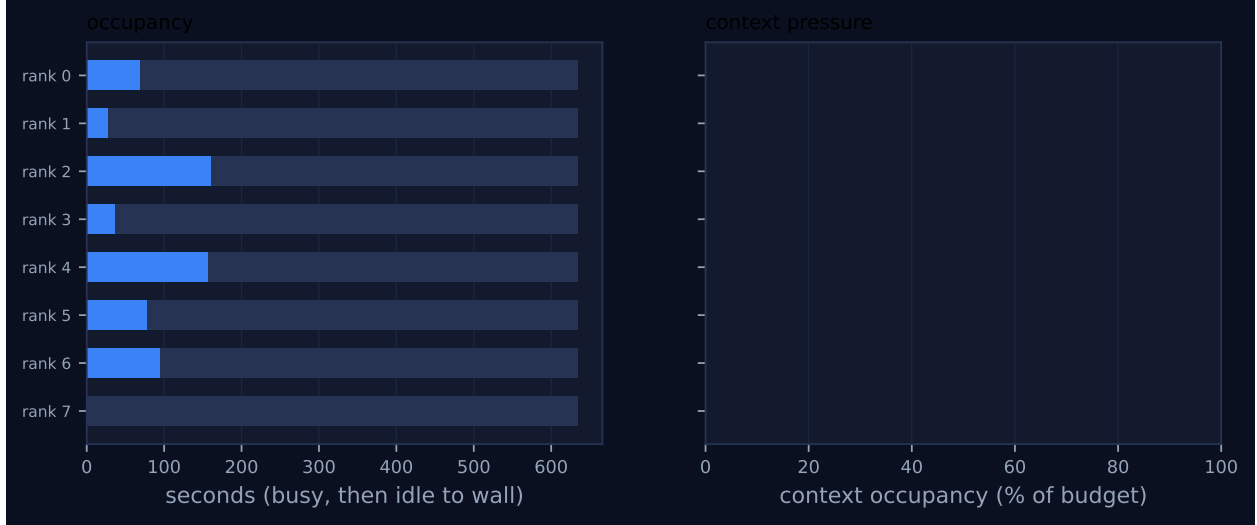


Figure 3: Per-rank occupancy and context pressure. Left: busy time, with the remainder to wall time in grey. Right: context occupancy against budget, amber above half and red above 80%, where admission pressure starts to reject artefacts.

4 Per-rank detail

rank	busy (s)	occ. (%)	tasks	calls	sent	recv	tok. sent	ctx (%)	state
0	68.99	6.9	1	1	0	1	0	0.0	finalized
1	27.76	3.4	1	1	1	1	315	0.0	finalized
2	161.63	17.6	1	1	1	1	453	0.0	finalized
3	36.78	4.3	1	1	1	1	315	0.0	finalized
4	157.38	15.7	1	1	1	1	464	0.0	finalized
5	77.86	8.5	1	1	1	1	421	0.0	finalized
6	94.52	10.4	1	1	1	1	307	0.0	finalized
7	0.00	0.0	0	0	1	0	173	0.0	finalized

Per-rank behaviour. Occupancy is busy time over the rank’s own lifetime, so a rank that joined late is not penalised for the time before it existed.

5 Collectives, against the cost model

op	algorithm	label	p	seen	rounds	pred.	msgs	pred.	tokens	wall (s)	skew (s)	model
reduce	chain	—	8	8	7	7	7	7	2,343	634.43	634.43	✓

Messages are the count the fabric logged, attributed to each invocation through its internal tag, rather than the count the collective reported — the two are independently produced, and preferring the log is what makes this a check rather than a restatement. A dash in the model column means the comparison is not meaningful: either no closed form exists for that algorithm, or the invocation never completed.

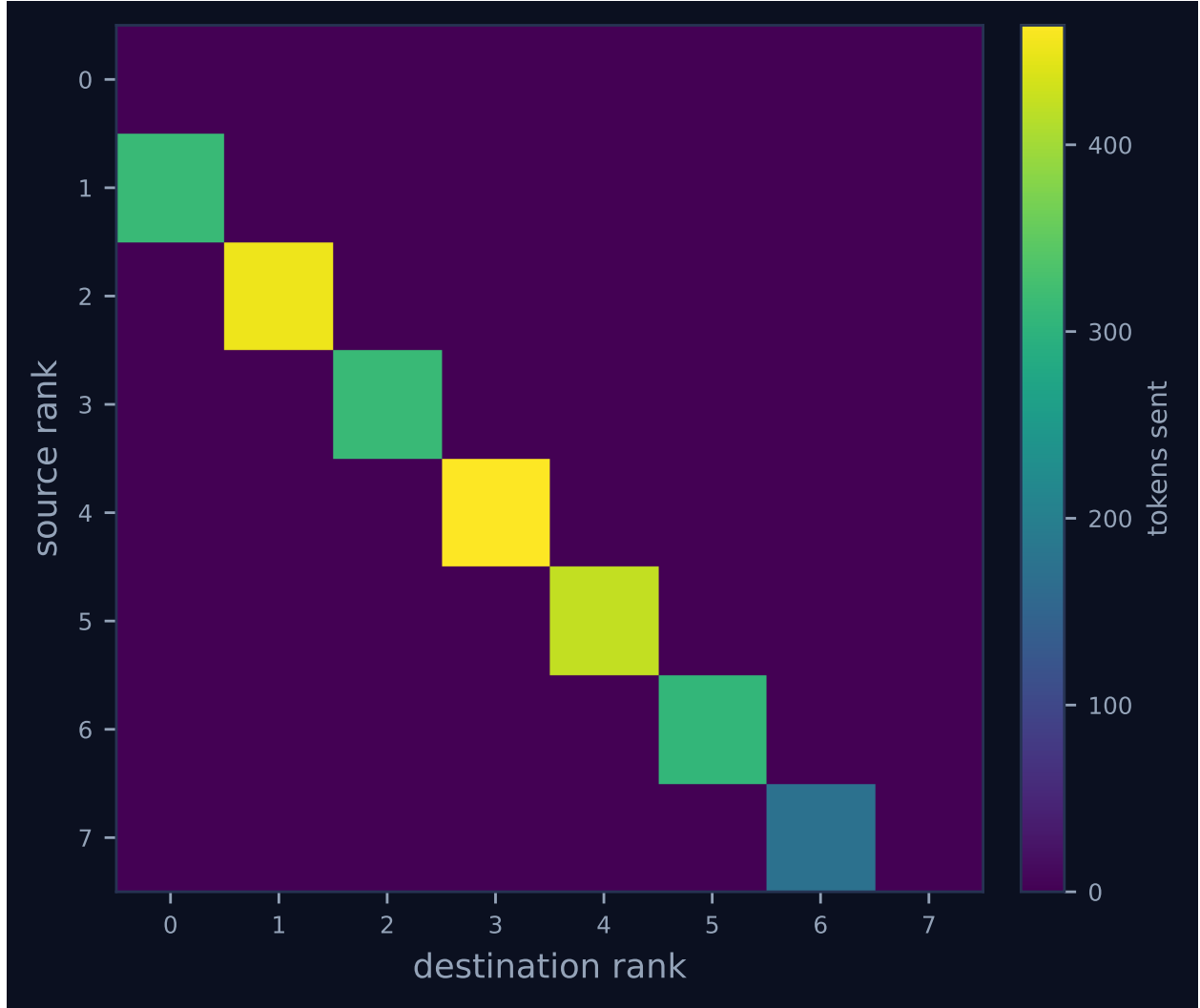


Figure 4: Communication matrix: tokens sent from each rank to each rank. The algorithm’s shape is visible here — a tree concentrates traffic on a root, a ring forms a diagonal band, and an unintended all-to-all fills the plane.

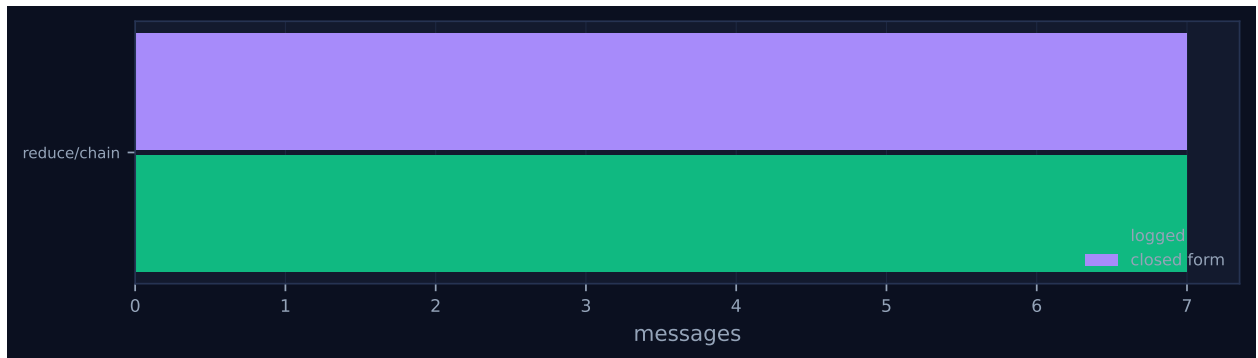


Figure 5: Logged message count against the closed-form prediction, per invocation. A red diamond marks a disagreement between the implementation and its own cost model.

6 Reading the timeline

Figure 1 is a descending staircase and nothing else. Rank 7 emits one tick at $t = 0.01$ s and never appears again; rank 6 holds a bar, hands off to rank 5, which hands to rank 4, and so on up to rank 0, whose bar ends at 634.44 s. Exactly one rank is busy at any instant — peak ranks busy 1 of 8, serial fraction of active time 100% — which is the algorithm’s definition rather than a defect: rank r blocks in `_crecv` until rank $r + 1$ delivers its accumulator, folds its own payload in as the left operand so that evaluation order is exactly rank order, and sends onward. The `coordination_share` of 53.3% is the highest of the four cells for the same reason: seven ranks spend most of the run blocked inside the collective, and those rank-seconds are counted, where the flat cell’s seven idle ranks return in milliseconds and contribute nothing to the same statistic. Load imbalance of $2.07\times$ is the flattest in the campaign, and is the one respect in which the chain looks good: it spreads its work over seven ranks instead of one.

The bars are wildly uneven and this is the run’s main measurement trap. The seven merges took 94.52, 77.86, 157.38, 36.78, 161.63, 27.76 and 68.99 s, a $5.8\times$ spread, on prompts of 446, 577, 691, 735, 585, 723 and 585 tokens — a range of $1.6\times$ with no correlation to latency. Almost none of this is queueing: differencing every `broker.enqueue` $\rightarrow$ `broker.claim` pair in the log gives 7.1 s of total claim wait against 624.9 s of generation, so the 634 s is real model time. But it is real model time at 89.3 s per call against the binomial sibling’s 21.8 s for prompts of the same size and outputs of the same size, which is a property of the worker pool at the moment this cell ran, not of the collective. The mechanical prediction is that seven serial applications cost 3.5 times what the k -ary tree’s two cost; the measured ratio is $12.3\times$, and the difference between those two numbers is pool-side. The log runs to 1001 s, 367 s past `job.finish`, with 47 reattachments and 24 trailing events, and the dashboard’s health table shows all eight ranks red and suspected `fail_stop` for that reason while `metrics.json` records `ok: true` and no failures.

The communication matrix, Figure 4, is the chain’s superdiagonal: seven edges $7 \rightarrow 6, 6 \rightarrow 5, \dots, 1 \rightarrow 0$ carrying 173, 307, 421, 464, 315, 453 and 315 tokens, 2,448 in total — the most fabric traffic of the four cells, because every hop ships an accumulator rather than a leaf. The weights are the fidelity story in profile: they rise from 173 to 464 as the accumulator fills and then oscillate between 315 and 453 instead of continuing to rise. An accumulator that stops growing at hop four of seven is an accumulator that has stopped accumulating.

7 Interpretation

The structure is by construction and the checks pass: seven messages against a closed form of $p-1 = 7$ and seven rounds against $p-1 = 7$, so `model_checks_agree` is 1 of 1 and Figure 5 shows no red diamond. The regime must be established before anything is read into the retention figure, and the fabric settles it. All seven `agent_calls` rows carry `contract` with `max_tokens: 450` and the non-empty semantics clause instructing the operator to “omit whole items rather than abbreviating or re-encoding any”. This is the enforced regime: the runtime counts the returned tokens, rejects an overrun and retries. The `sat-mb-fidelity` chain ran the same p , the same twelve items per rank and the same nominal 450-token budget with `max_tokens: null`, emitted 695 tokens at the root, and scored 1.000. The provenance commit `f121647` does not contain `merge_contract`; the enforced version was committed two minutes after this campaign wrote its results as `561d8a17`, so the commit field records `HEAD` at launch and the fabric is the authority on what actually ran.

What emerged is the fold’s internal shape, and it separates two things the aggregate conflates.

Reconstructing every intermediate from the store gives per-rank item counts of $\{6:12, 7:12\}$ at depth 1; $\{5:12, 6:12, 7:10\}$ at depth 2; $\{4:12, 5:12, 6:12, 7:2\}$ at depth 3; $\{3:12, 4:12, 5:1\}$ at depth 4; $\{2:12, 3:12, 4:12, 5:1\}$ at depth 5; $\{1:12, 2:12, 3:1\}$ at depth 6; and $\{0:12, 1:12, 2:12, 3:1\}$ at depth 7. Every one of those is a strict prefix of the rank-ordered item list: the lowest-numbered ranks present in the input are kept whole, the tail is shaved, and nothing in between is thinned. That is whole-rank-plus-partial-tail erasure rather than under-representation, it explains `rank_position_correlation` = -0.863 , and it is a direct consequence of two independent decisions — the operator writes items in the order it reads them, and `reduce` makes the local payload the left operand so that a non-commutative operator sees rank order. Also worth recording: `n_label_only` is 0, so `value_retention` and `id_retention` are both 0.3854 exactly. Every identifier that survived carried its correct six-digit serial and four-letter checksum. The operator is honest; it simply stops.

Where this cell disagrees with its siblings is on whether the budget explains the loss, and the disagreement is the most useful thing here. An incompressible item is `[F-0-0] serial 597623 checksum LVPG`: nine tokens of content, 11.81 tokens once quoted into the JSON array, so a 450-token ceiling holds 38.1 items. The tree cells sit exactly there — 437 tokens for 37 items, 449 for 38 — and their loss is the ceiling. This run’s outputs oscillate: 449 tokens and 38 items when the input fits, then 300 tokens and 25 items when it does not, then 438 and 37, then 300 and 25, then 438 and 37. The applications that are forced to drop undershoot the ceiling by a third and delete about thirteen items more than they had to. So the chain’s loss is partly the budget and partly the operator being more conservative than instructed, and the two cells’ agreement on 0.385 is arithmetic coincidence: the last application here received $25 + 12 = 37$ items, which fits, so it lost nothing, and 37 is also where the tree cells’ ceiling lands. Read carefully, this strengthens rather than weakens the campaign’s conclusion — depth did not cost this run anything even though it had seven chances to compound loss — but it means “the same 37 survivors under every algorithm” should not be read as a single mechanism operating four times.

Of the flagged findings, the serial-execution warning is the algorithm’s definition, the $2.07\times$ imbalance is a note and is genuinely the best in the campaign, and the 47 reattachments up to incarnation 13 are the broker’s normal mode under a shared pool. None is a defect. This cell’s contribution relative to its siblings is to bound the cost of depth: seven applications on the critical path, 634.43s against the k -ary tree’s 51.77s and the binomial tree’s 80.27s, for exactly the same 37 items and the same 2,623 output tokens. It is the run in the campaign whose result most cleanly says that nothing is bought by folding deeply.

8 Threats to this reading

The executor is real — `executors` records `broker` $\times 8$ and `worker` $\times 47$ — so this is model behaviour, not a surrogate implementing a loss model by construction as `fid-synth` and `fid-inc-smoke` do. The latency claim is the one I would most want to withdraw. With one trial per algorithm I cannot separate the chain’s 89.3s mean call latency from run-to-run variability in the pool, and this cell’s own calls range over 27.76–161.63s on near-identical prompts, so the within-run standard deviation is already large enough to make a between-run comparison unsafe. Treat the direction as established by mechanism — seven serial applications cannot be faster than two — and the magnitude, $12.3\times$ against k -ary, as an overstatement of roughly $3.5\times$ worth of structure plus an unknown amount of pool weather.

Second, the intermediate reconstruction depends on the accumulators in `runs/inc-mb-fidelity-inc/fidelity-ch` being the objects the operator actually saw, which they are: each is the `result_digest` of one

`agent_calls` row and the `digest` of the corresponding `msg.send`, so the chain of custody from one application to the next is in the fabric. Unlike its siblings this cell shares no result digest with any other run in the campaign — its fold order is the reverse of the flat cell’s and its intermediates are disjoint from the tree cells’ — which incidentally removes the shared-pool convergence question here almost entirely. The one shared prompt digest, `1586acb2d322` between this run’s `merge:d1` and the binomial cell’s, returned different results (292 against 294 tokens), so the executor is demonstrably non-deterministic and no identity in this campaign can be explained by replay.

Third, the 11.81-tokens-per-item constant comes from the repository’s approximate counter with `tokenizer_exact: false`. That is the same counter the contract check uses, so the arithmetic is internally consistent, but the “38 items fit” figure is not a claim about any production tokeniser. The \$0.0524 is modelled from \$3/Mtok in and \$15/Mtok out. And `fact_retention` counts an identifier wherever it occurs in the result text, including the merge prompt’s own instruction line “such as [F-3-2]”, which lies inside the benchmark’s identifier space; I checked, and rank 3’s survivor here is `F-3-0` at every depth, so no phantom inflates this run — but it does inflate the surrogate campaigns and the metric should exclude the template.